BuildCost MVP — REVISED EXECUTION PLAN v2

TEAM ASSIGNMENTS

Workstream	Owner	Why
WS1: SQM Engine (surface calculation from plans)	**Tiemen**	Primary, semi-independent project. Most technical/geometric challenge.
WS2: AI Pipeline (QQP discovery, finishing levels, cost calc)	**Georges**	Domain logic, reference dossier analysis, pricing model.
WS3: Frontend & Infra	**Split** (see below)	Parallelizable UI work.

WS1: SQM ENGINE (Tiemen)

> **Mission:** Given any plan (PDF, image, CAD), output precise room-by-room m² and total building m². This is a standalone module that the rest of the system consumes.

Why this is separate:

- It's the hardest technical problem (scale detection, OCR, geometry)
- It needs to work independently of the QQP/pricing logic
- It needs dedicated prompt engineering and validation
- It's the #1 credibility factor for insurance companies

Key challenges to solve:

1. **Scale detection** — find scale bar, or calibrate from known dimensions (standard door = 80cm, standard wall thickness)
2. **Room boundary detection** — identify walls, room labels, dimensions
3. **Multi-format support** — PDF plans, scanned images, photos of plans
4. **Multi-floor handling** — detect pages = floors, aggregate correctly
5. **Sub-area classification** — living vs utility vs circulation vs outdoor
6. **Belgian plan conventions** — room naming (NL/FR), dimension formats

Output contract (API):

```json
```

```
{
  "plan_id": "uuid",
  "scale_detected": true,
  "scale_method": "scale_bar | door_calibration | dimension_text | user_input",
  "confidence": 0.87,
  "floors": [
    {
      "level": 0,
      "label": "Gelijkvloers",
      "rooms": [
        {
          "id": "r1",
          "label": "Living",
          "label_original": "Leefruimte",
          "area_sqm": 42.5,
          "dimensions": {"length": 8.5, "width": 5.0},
          "category": "living",  // living|bedroom|bathroom|kitchen|utility|circulat
          "floor_level": 0,
          "features": ["large_window", "fireplace"],
          "confidence": 0.92
        }
      ],
      "total_sqm": 145.3,
      "circulation_sqm": 12.4
    }
  ],
  "summary": {
    "total_livable_sqm": 245.6,
    "total_utility_sqm": 35.2,
    "total_garage_sqm": 28.0,
    "total_outdoor_sqm": 15.0,
    "total_gross_sqm": 308.8,
    "floor_count": 2,
    "has_basement": false,
    "has_attic": true
  }
}
```

Tiemen's timeline:

Hours	Task
0-2	Setup with Georges (repo, Supabase, Vercel)
2-4	**Benchmark LLM vision APIs** on 5 real plans — Claude Sonnet, Claude Opus, GPT-4o. Compare extraction accuracy. Pick best.

Hours	Task
4-8	**Build extraction prompt v1** — iterative prompt engineering for room detection, dimensions, labels. Test on 10+ plans.
8-12	**Scale detection module** — multiple strategies: scale bar OCR, dimension text parsing, door-width calibration. Fallback chain.
12-16	**Area calculation engine** — from extracted dimensions to precise m². Handle irregular rooms, L-shapes, annotations.
16-20	**Multi-floor & multi-format** — PDF page splitting, image preprocessing, floor aggregation logic.
20-24	**Validation framework** — compare extracted m² vs known m² from reference dossiers. Track accuracy %.
24-28	**API endpoint** — `/api/extract-sqm` callable by Georges' pipeline
28-32	**Accuracy hardening** — fix worst cases, improve prompts, add preprocessing
32-36	Integration with main app + testing with real plans

WS2: AI PIPELINE (Georges)

> **Mission:** From extracted plan data (consuming WS1 output), discover QQPs, determine finishing level, calculate rebuild cost.

Scope:

- QQP definition & seeding
- Reference dossier ingestion pipeline
- QQP extraction from plan data
- Correlation analysis (QQP ↔ price/m²)
- Finishing coefficient calculation
- Self-learning loop
- Cost calculation (base price × ABEX × finishing coeff)
- Postcode price table + ABEX import

Georges' timeline:

Hours	Task
0-2	Setup with Tiemen (repo, Supabase, Vercel)

Hours	Task
2-6	**Seed QQP definitions** (20-30 initial params) + import postcode prices + ABEX index into Supabase
6-10	**Reference dossier pipeline** — upload flow for plan + known price. Store in Supabase. Trigger extraction.
10-14	**QQP extraction prompt** — given structured plan data (from WS1), extract QQP values. Test on reference dossiers.
14-18	**Correlation engine** — statistical analysis: which QQPs predict price/m² best? Assign initial weights.
18-22	**Finishing coefficient model** — from QQP values → single coefficient (0.7 to 1.5). Map to categories (Basic→Premium).
22-26	**Cost calculation endpoint** — full formula: m² × base_price × ABEX × finishing_coeff
26-30	**Self-learning loop** — process 50+ reference dossiers, iterate weights, discover new QQPs
30-34	**Integration with WS1** — end-to-end: plan upload → SQM extraction → QQP analysis → cost
34-38	Accuracy validation against known prices, tune model

WS3: FRONTEND & INFRA (Split)

TOGETHER (Hours 0-2): Foundation

- [ ] Create GitHub repo (monorepo: Next.js)
- [ ] Create Supabase project → run migrations
- [ ] Create Vercel project → connect to GitHub
- [ ] Set up Supabase Storage buckets (plans, reference-dossiers)
- [ ] Configure environment variables
- [ ] Deploy skeleton app

GEORGES handles (between AI pipeline work):

Hours	Task
6-8	**Supabase Auth** — magic link signup, tenant creation on first login
14-16	**Admin: reference dossier upload UI** (needs this for own pipeline testing)
26-28	**Results page** — cost breakdown, QQP visualization, finishing level gauge

Hours	Task
34-36	**Dashboard** — estimation history, model accuracy charts

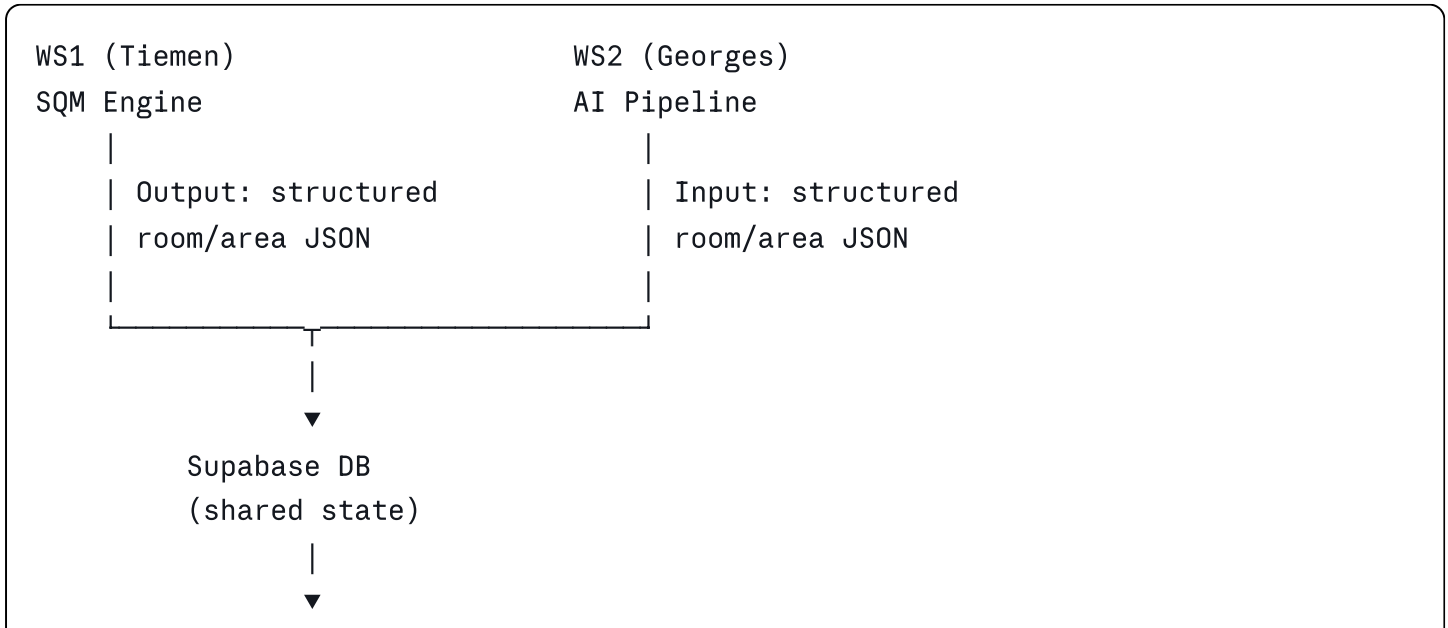
TIEMEN handles (between SQM work):

Hours	Task
4-6	**Landing page** — hero, value prop, CTA, trust signals
12-14	**Upload flow UI** — drag & drop, file validation, postcode input, processing status
20-22	**SQM results display** — room breakdown table, floor summary, area visualization
28-30	**Mobile responsive pass** — test and fix all pages on mobile

TOGETHER (Hours 36-48): Integration & Polish

Hours	Task
36-40	Wire everything end-to-end, fix integration bugs
40-44	Process bulk reference dossiers, validate accuracy
44-46	Edge cases, error handling, loading states
46-48	Demo prep, seed demo data, rehearse

INTEGRATION POINTS



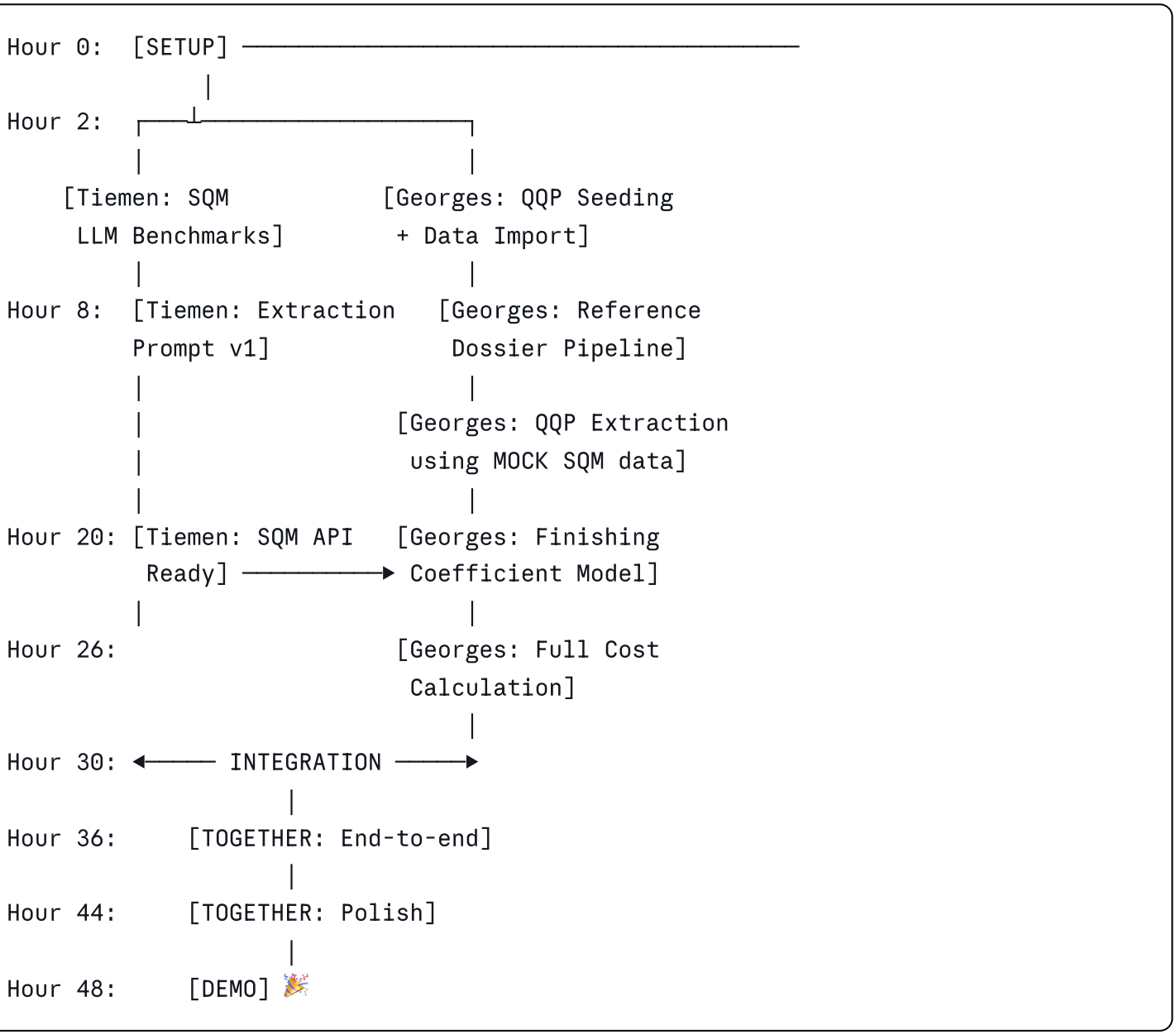
```
          WS3: Frontend
          (both contribute)
```

Critical handoff: Tiemen's SQM extraction JSON format (defined above) is the contract. Georges builds the QQP engine to CONSUME that format. They can work in parallel as long as this contract is agreed upon.

Mock data strategy:

- Georges can start QQP work with **manually created** SQM JSON (from reading plans by eye) while Tiemen builds the automated extraction
- Tiemen can test SQM extraction independently without needing the pricing pipeline
- Both converge at hour 30-34

DEPENDENCY GRAPH



IMMEDIATE NEXT STEPS (Right Now)

1. **Both:** Do we agree on this split? Any adjustments?
2. **Both:** Set up repo + Supabase + Vercel (30 min)
3. **Tiemen:** Share 5 diverse reference plans (different types, qualities)
4. **Georges:** Share postcode price list + ABEX file + 5 reference dossiers with known prices
5. **Both:** Confirm the SQM output JSON contract above works for both sides

Once confirmed, I'll generate:

- The Supabase migration SQL (ready to run)
- The Next.js boilerplate with project structure
- The initial SQM extraction prompt for Tiemen to test
- The QQP seed list for Georges to start with